HW4: Kalman Filter

Problem

A delivery platform uses GPS to track the locations of its delivery drivers. However, GPS measurements may contain noise or become temporarily unavailable, making it difficult to accurately estimate their positions.

In this assignment, **you will implement a Kalman Filter** to estimate the positions of delivery drivers using GPS measurements. **The assignment consists of four tasks,** starting with basic tracking and progressively introducing more challenging scenarios, including GPS signal loss, multiple delivery drivers whose speeds change over time, and changes in movement direction.

Task 1: Basic Tracking (20 points)

- Scenario: One delivery driver moves in a straight line at constant velocity. GPS measurements are available every second without interruption.

Task 2: GPS Loss (20 points)

- Scenario: One delivery driver moves in a straight line at constant velocity. GPS becomes unavailable once for 5–10 seconds.

Task 3: Multiple Drivers (30 points)

- Scenario: Three delivery drivers move independently in straight lines with different directions. Their speeds change gradually from time to time. Each driver experiences one GPS signal loss lasting 2–5 seconds.

Task 4: Turning Drivers (30 points)

- Scenario: Five delivery drivers move at constant speed but change direction along their routes. Each driver experiences two GPS signal losses, each lasting 2–5 seconds.

Assignment Requirements

This is an individual assignment. All work must be completed independently.

Required programming environment: Python 3.11, with numpy and filterpy (see requirements.txt).

Four Python templates (task1.py, task2.py, task3.py, and task4.py) are provided, one for each task. The GPS datasets and evaluation program (eval.py) are also provided.

You only need to complete the Kalman Filter implementation in these four templates. Data loading and evaluation are handled by eval.py.

All templates use a 2D constant-velocity Kalman Filter to estimate the position and velocity of each delivery driver.

The state vector is defined as:

$$\mathbf{x} = [x, y, v_x, v_y]^T$$

where x and y represent position in meters, and v_x and v_y represent velocity in meters per second. The time interval between consecutive time steps is 1 second.

Two matrices are already provided in the templates and must not be modified:

- State Transition Matrix (**F**): A 4×4 matrix that describes how position and velocity change between time steps.
- Measurement Matrix (**H**): A 2×4 matrix that relates the state vector to GPS measurements, which provide position information but not velocity.

You need to complete the following three parts in each template:

1. make_matrices() — Define Covariance Matrices
   Define the following three matrices used by the Kalman Filter:
   - Initial State Covariance Matrix ($\mathbf{P_0}$): A 4×4 matrix that represents uncertainty in the initial position and velocity estimates.
   - Process Noise Covariance Matrix (**Q**): A 4×4 matrix that represents uncertainty in the constant-velocity motion model.
   - Measurement Noise Covariance Matrix (**R**): A 2×2 matrix that represents uncertainty in the GPS position measurements. The GPS measurement noise has a standard deviation of approximately 5 meters along each axis.

   You may adjust $\mathbf{P_0}$ and **Q** for different tasks, while keeping **R** fixed.

2. Tracker.__init__(driver_ids) — Initialize the Tracker
   The Tracker class is responsible for tracking delivery drivers. The parameter driver_ids contains the IDs of the drivers to be tracked.
   Create a separate Kalman Filter for each driver and configure it using the provided matrices **F** and **H**, together with $\mathbf{P_0}$, **Q**, and **R** defined in make_matrices().

3. Tracker.step(observations) — Estimate Positions
   This method estimates the current position of each delivery driver using GPS measurements. The parameter observations is a dictionary in the form $\{\text{driver}_{\text{id}}: (\text{gps}_x, \text{gps}_y) \text{ or None}\}$, where each key is a driver ID and each value is the current GPS position or None if GPS is unavailable.

   Implement the Kalman Filter tracking process. On the first call (t = 0), initialize each driver's position using the GPS measurement and set the initial velocity to zero. At subsequent time steps, perform Prediction followed by Update when GPS is available. If GPS is unavailable, perform Prediction only.

   Return a dictionary in the form $\{\text{driver}_{\text{id}}: (\text{estimated}_x, \text{estimated}_y)\}$ containing the estimated position of every driver at each time step, including those with missing GPS measurements.

After completing your implementation, use eval.py to test your Tracker on the public train and validation datasets. The program reports the Root Mean Squared Error (RMSE) for the selected task(s) and data split(s).

COMMAND	DESCRIPTION
python eval.py --task 1	Evaluate Task 1 on both train and validation data.
python eval.py --task all	Evaluate all four tasks on both train and validation data.
python eval.py --task 3 --split val	Evaluate Task 3 on validation data only.
python eval.py --task 2 --split train --save pred.csv	Evaluate Task 2 on train data and save per-second predictions to pred.csv.

Each task is evaluated using RMSE on the hidden test dataset. Different tasks have different full-

score and zero-score thresholds.

- Full Score: RMSE at or below the full-score threshold.
- Zero Score: RMSE at or above the zero-score threshold.
- Partial Score: Scores are calculated by linear interpolation between the two thresholds.

Dataset

The task of this dataset is to estimate each delivery drivers' true position from noisy, sometimes missing GPS measurements.

The dataset is in the data/ folder of the assignment package. All files are .csv: the first line holds the column names, and each following line is one driver at one second. Each scenario lasts 60 seconds (t = 0 – 59).

Each task (taskN = task1 – task4) has three splits: train, validation (val) and hidden Test.

- taskN/train.csv, taskN/train_gt.csv – input and ground truth of the train split: 30 scenarios
- taskN/val.csv, taskN/val_gt.csv – input and ground truth of the val split: 30 scenarios
- hidden Test – kept by the TAs and used for grading: 60 scenarios

The columns are as follows:

Column	Type	Description
scenario_id	string	Scenario ID
time_s	int	Time in seconds, from 0 to 59
driver_id	int	Driver ID (1, 2, 3, ...)
Measured_X	float	GPS X position (m); NaN when GPS is missing. Available in train.csv / val.csv only.
Measured_Y	float	GPS Y position (m); NaN when GPS is missing. Available in train.csv / val.csv only.
true_x	float	True X position (m). Available in train_gt.csv / val_gt.csv only.
true_y	float	True Y position (m). Available in train_gt.csv / val_gt.csv only.
true_vx	float	True X velocity (m/s). Available in train_gt.csv / val_gt.csv only.
true_vy	float	True Y velocity (m/s). Available in train_gt.csv / val_gt.csv only.

Submission Instructions

Submit one ZIP file named: StudentID_name.zip

Your ZIP file must contain the following four files:

```
StudentID_name.zip
├── task1.py
├── task2.py
├── task3.py
└── task4.py
```

Make sure your program runs successfully and all required files are included.

Submissions that cannot be graded due to missing required files or execution errors may receive a score of 0.

Any confirmed plagiarism will result in a score of 0 for the entire assignment.

Deadline

November 12, 2026, at 23:59 (local time). **Late submissions will not be accepted.**